

Post-Quantum Cryptography

The University of Texas at Dallas
Fun(key)

Ali Sbeiti – Ali.Sbeiti@utdallas.edu
Troy Frazier – trf170130@utdallas.edu
Samantha Gant – Samantha.Gant@utdallas.edu
Aviral Verma – Aviral.Verma@utdallas.edu

December 2019

Contents

Project Summary	2
Description	2
Motivation	2
Organization	3
Deliverables	3
Budget	3
Schedule	3
Approach and Justification	3
Results	4
Packet Fragmentation	4
SSL/TLS Key Exchange	4
Post-Quantum VPN	5
Time Analysis of Key Transmission	6
Bit Error Rate and Large Keys	7
Future of SSL/TLS	8
Benchmarking of Post Quantum Cryptography Algorithms	10
Literature Review	13
Introduction	13
Quantum Computing	13
Public Key Distribution and Coin Tossing	13
Post-Quantum Key Exchange	14
Crypto-agility Strategies	14
Cryptography: Union of Cryptography and Stenography	15
References	16
Investigator Biographies	17
Ali Sbeiti	17
Troy Frazier	17
Samantha Gant	17
Aviral Verma	17

Project Summary

Description

Efforts have begun to develop quantum-resistant cryptographic algorithms as the threat quantum has on present algorithms is great. NIST is holding a round 2 of standardization to help address additional problems besides the mathematics.

The goal of this project is to find niche problems that other groups have yet to research and find interesting differences between what is known versus the special cases. There are 3 candidate cases, implementation practicality, benchmarking on unusual systems, and implementation comparison.

Motivation

Our common interests lie within Cryptography so we are interested in the project. Our members cryptography skills range from beginner to advanced and we all desire to learn more. One of our members has gotten training with IBM for cryptography, and a couple of our members learn cryptography as a hobby. We all know the foundations of cryptography, like public/private key encryption, hashes, and the common attacks like bruteforce, man-in-the-middle, and meet-in-the-middle. This project is exciting to us as the invention of quantum is both a great advancement and a great threat to cryptography and we want to mitigate the threat.

Organization

Deliverables

- Proposal and Literature Review
- Progress Report and Presentation
- Final Report and Presentation
- Documentation and Code Delivery

Budget

We don't currently anticipate any financial costs to this project. The only potential variance would be the need for non-open source protocols or for purchase of a machine with greater computing power to test our theories. However, neither of these is currently expected to be necessary.

Schedule

- October 4: Proposal due
- October 11: Determine Official Research Area
- October 25: Finalize Resources for Midterm
- November 1: Midterm Report due
- November 8: Formalize Direction Corrections
- November 22: Finalize Resources for Final
- December 6: Final Deliverable due

Approach and Justification

The focus of our research will be looking at the feasibility of implementation of mathematically proven methods of post-quantum cryptography. This will involve implementing approaches such as addition of quantum keys with existing certificates and protocols to determine what the current response to these methods is, namely whether they are properly handled, cause a crash, or cause improper fragmentation.

Our research will involve throwing large, quantum-sized, keys with arbitrary data at various servers using existing proven-secure connection methods to gauge response and determine what qualifies as too large. Significant responses being watched for will be improper fragmentation and the possibility of buffer overflow.

Results

1 Packet Fragmentation

Currently we are in the transition process between IPv4 and IPv6. For RSA specific keys, NIST proposes that 4096 bits are considered “Quantum-Safe”. Under the size of a IPv4 packet. 4096 bit keys can still fit in a single packet. However, some networks may be limited to a size smaller than 4128 (IP header + key size). In this case, key fragmentation will still occur. In small cases where the fragmentation is only a small amount of fragments, there is not a huge security risk. However, if there are hundreds and even thousands of fragments, then factors like network noise and TCP attacks become more potent. Quantum keys will continue to grow larger so the threat of fragmenting keys in IPv4 is an extreme concern

A solution is within IPv6 packets. IPv6 do not fragment as IPv6 header information does not contain fragmentation information. However, IPv6 is not fully implemented as will require overhauls in both networks and hardware which will pose a challenge to actually fully implement. Another proposed problem is the fact that a sender could fragment the packets themselves by simply breaking their data into chunks and send those as full IPv6 packets which would make the solution of fragmentations redundant. The solution relies on the user to be secure which is not a viable security concept. On top of that, while IPv4 can handle current RSA quantum-safe keys, key sizes are expected to continue to grow so IPv6 is only a temporary solution.

Assuming a large amount of fragmentation and the possibility of a noisy network, TCP is the obvious choice. Some protocol like IKE use UDP to transfer keys but given a large amount of fragmentations and the importance of getting the full key, TCP is needed. However, such an implementation opens up to TCP attacks like packet injection. Assuming a perfect attack, as well as the strength of quantum computing to guess the correct TCP packet, we can effectively take over the communications between two points and succeed in a Man-in-the-Middle attack. That is just one example of an attack, as many others like disrupting sessions will be easy enough to attempt. Such attacks will be also affect RSA and will need to be tested against.

2 SSL/TLS Key Exchange

In normal cases, when a client attempts to authenticate a server, the server will respond with its certificate, which includes the server’s public key. This certificate will be used by the client to verify the server’s identity. Post authentication, the public key of the server gets used to distribute a shared symmetric key so that data sent between the client and server can be encrypted. During the initial asymmetric key distribution, NIST recommends that servers use 2048-bit RSA keys in the certificate to keep data confidential for the near future. Current trends in many services and cipher libraries suggest that the 2048-bit key standards has already been widely adopted and implemented. However, NIST also recommends an asymmetric key size of 4096-bits or greater to be “Quantum-Safe”. This could

detrimental to existing secure systems if the development of quantum computing progresses faster than anticipated.

Our primary objective is to find any performance impacts of the distribution, signing, and verification of SSL/TLS certificates with larger key sizes. Starting at NIST's "Quantum-Safe" 4096-bit keys to even larger key sizes, measuring the average penalties in distribution speed compared to modern 2048-bit standards. We will also look into how much more computationally intensive the distribution of these larger keys are when a server has to respond to many clients at once, simulating the impact these larger keys may have on many real world services. Another pipelined objective is to find which existing services allow clients to request greater than 2048-bit RSA keys for the initial asymmetric handshake.

3 Post-Quantum VPN

It is a widespread notion among security groups that RSA keys will be rendered unsalvageable with the emergence of quantum computers. In reality however, breaking RSA key size of 2048 bits requires 4000 qubits estimatedly. By contrast, current quantum computers are pushing 72 qubits, such as Google's Bristlecone design. If those qubits were perfect and required no error correction (i.e. a 1:1 logical to physical ratio) then the 2048 bit number could be factored as early as 2023. However, if heavy error correction is required with a 1000:1 logical to physical ratio, then this factoring could not be accomplished until about 2041 or about 18 years later. However it is without doubt that such a quantum computer will be built soon. For now implementing key sizes of 3072 bits or larger gives us an ample amount of time while those of 4096 bits are essentially "quantum safe".

Creating a VPN in Post Quantum world for now could rely on RSA handshake with key size of 4096 bits in order to be quantum resistant. Although this solution is just temporary. There are alternative options available that promise to deliver a more durable solution. Microsoft's Post Quantum Cryptography VPN project for example is based on three PQCrypto algorithms submitted to NIST. Frodo, a key exchange protocol based on the learning with errors problem; Sike, a key exchange protocol based on Supersingular Isogeny Diffie-Hellman and Picnic, a signature algorithm using symmetric-key primitives and non-interactive zero-knowledge proofs. They also provide software and instructions for building a post-quantum secure VPN appliance with a Raspberry Pi 3. The device acts as a WiFi access point, and tunnels all of its traffic over the post-quantum VPN.

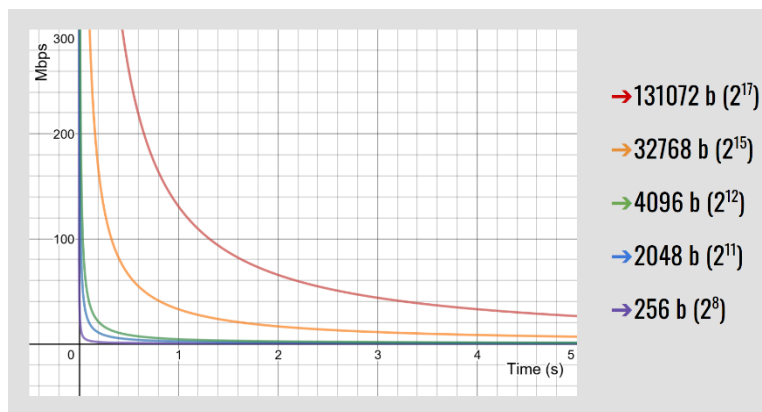
Our objective is to look into security measures essential in the immediate future. We intend to analyse and possibly compare the performance and practicality of these two handshake approaches and answer the question, whether for now is it more efficient to stick to increasing key size of RSA or switch to VPNs relying on Post Quantum Cryptography algorithms.

4 Time Analysis of Key Transmission

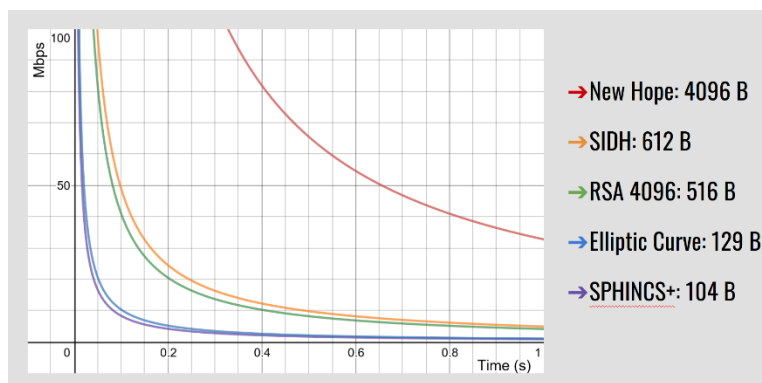
To analyze the time analysis of transmitting various RSA key sizes, we graphed internet speed, in Megabits per second (Mbps) versus time, in seconds. For our range of speeds, we used 100 Mbps as the low end, since tests showed 60-100 Mbps as the standard for personal networks, and 300 Mbps as the high end, as this was the speed we found through university networks. Our formula was:

$$f(x) = \frac{1000(\text{key size in bits})}{1000000 x} = \frac{(\text{key size in bits})}{1000 x}$$

The key sizes we compared were 256-bit, the minimum accepted by any RSA implementation, 2048-bit, the current standard for RSA keys, and 4096-bit, the minimum required to be classified as quantum-safe. Additionally, we included key sizes of 2^{15} and 2^{17} to demonstrate the possibility and consequences of even larger key sizes depending on the capabilities of quantum computers once implemented. Specifically, our graph looks at the time needed to transmit 1000 keys of the specified size across various network speeds. From this graph, we see that up to 4096-bit keys have minimal impact on the time needed, but beyond that scope begins to produce substantial delays.

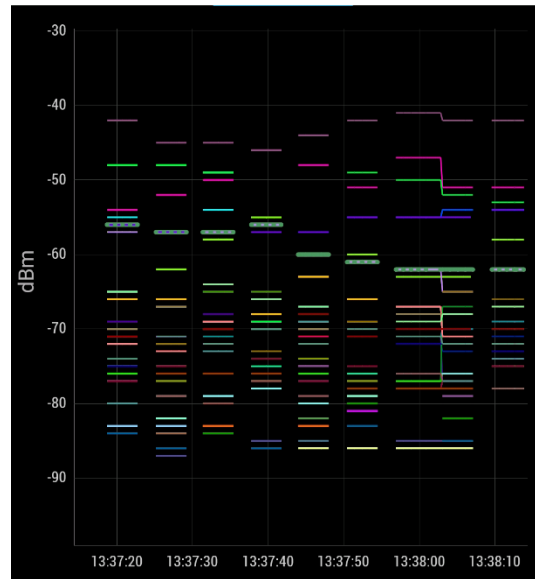


In addition, we compared RSA 4096 key size, since this is the quantum-safe size, to other post-quantum encryption algorithms. Namely, we compared New Hope and SPHINCS+, two NIST proposals, along with Elliptic Curve and SIDH. Again, we compared the algorithms' key sizes by looking at the time needed to send 1000 keys across various network speeds.

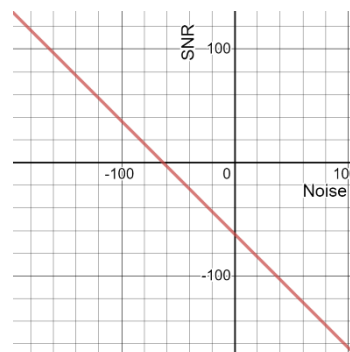
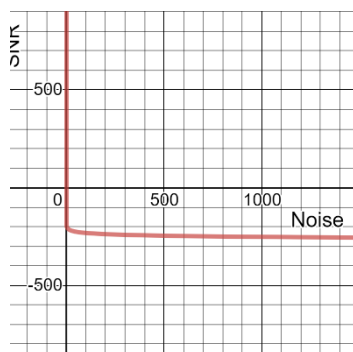


5 Bit Error Rate and Larger Keys

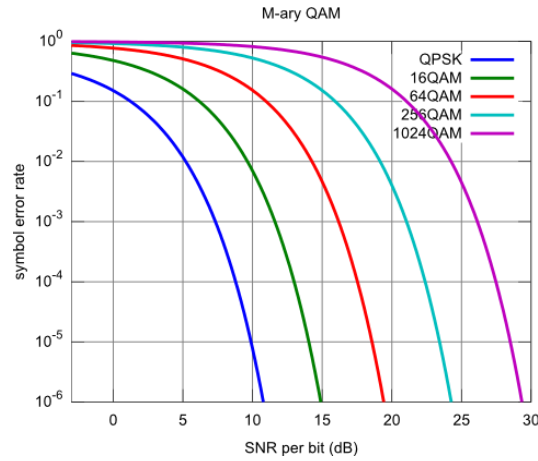
As keys get larger, the chances of bit errors grows. To start analysis of Bit Rate Error (BER), we began by measuring bandwidths of a large network through wireless hardware. Starting with our base measurements, we got approximately 20 devices, the average of average of all device's bandwidth was -64.3 Dbm, and download speed was 324 mbps. Taken into account was the average US mbps of approximately 100 mbps.



Now we were able to calculate Signal-to-Noise Ratio (SNR) which allows us to create a graph with noise as a variable. We will also assume that we are in an urban setting so a noise floor of -92 dbm will exist. Noise floor is the summation of all unwanted noise signals which changes depending on rural or urban settings. Urban cities have more noise so we will be measuring the worst case for analysis. Since we gathered our measurements through wireless media, we will be measuring BER with E_b/N_0 which compares BER and SNR. If we calculate SNR from the noise floor with our average bandwidth $(-64.3 + 92)$, we will get a SNR of 27.7 dbm. Now we can analyze a BER graph from cloudrf and we focus on 1024QAM modulation. At 27.7 we get a BER of 1 bit error for every 1,000,000 bits. Comparing that too 4096 bytes (32768 bits) which is the NIST current quantum resistant key size, we get a probability of 3.2768% chance of a bit error.



Now for the importance of BER within a quantum setting. Over time, we will continue to increase key sizes to keep up with the growing power of computing. Because of the noise floor, we will need to develop better modulations else the probability of BER will begin to negatively affect key transmissions. Because a packet is encrypted, we may not be able to repair packets from noise errors due to the need to decrypt and the avalanche theorem makes the smallest bit change a massive difference in decrypting. We could attempt to add error detection/repair outside of the encrypted packet but that would require changes within encryption mechanisms and increase complexity/overhead. In conclusion, BER is will definitely be a rising problem as we continue to increase key sizes.



6 Future of SSL/TLS

The rise of quantum computation poses a threat for many encryption algorithms in use today. Quantum systems have the potential of guessing keys a magnitude faster than conventional systems. This holds especially true for asymmetric algorithms. One solution might be to simply increase the key sizes of these asymmetric algorithms, making it much more unlikely that a quantum system can guess the key. One concern is the performance impact that increasing the key sizes might pose. To observe this, we generated a couple of n-bit sized (self-signed) TLS certificates and recorded the time it takes for the TLS handshake to complete.

1024-bit key	DNS Lookup	TCP Connection	TLS Handshake	Server Processing	Content Transfer
Test 1	4	72	128	54	1
Test 2	2	59	127	54	1
Test 3	4	61	126	52	0
Test 4	4	52	128	52	1
Test 5	4	84	129	51	1
Test 6	4	52	128	56	1
Test 7	2	56	131	55	1
Test 8	4	67	126	56	1
Test 9	4	52	127	53	0
Test 10	4	66	138	55	1
Average	3.6	62.1	128.8	53.8	0.8

2048-bit key	DNS Lookup	TCP Connection	TLS Handshake	Server Processing	Content Transfer
Test 1	4	85	124	56	0
Test 2	5	59	123	50	0
Test 3	4	62	126	56	1
Test 4	4	62	127	56	0
Test 5	3	59	122	55	0
Test 6	3	57	129	49	1
Test 7	4	56	135	52	0
Test 8	4	60	131	52	0
Test 9	4	53	126	51	0
Test 10	4	62	128	54	0
Average	3.9	61.5	127.1	53.1	0.2

4096-bit key	DNS Lookup	TCP Connection	TLS Handshake	Server Processing	Content Transfer
Test 1	4	191	141	54	1
Test 2	4	54	138	52	0
Test 3	4	59	130	54	0
Test 4	4	63	143	52	0
Test 5	4	61	134	50	0
Test 6	4	54	132	52	0
Test 7	4	61	138	56	1
Test 8	2	53	157	50	0
Test 9	4	54	137	57	1
Test 10	4	55	128	50	1
Average	3.8	70.5	137.8	52.7	0.4

8192-bit key	DNS Lookup	TCP Connection	TLS Handshake	Server Processing	Content Transfer
Test 1	4	61	216	56	1
Test 2	2	55	183	65	1
Test 3	4	65	184	56	0
Test 4	2	67	181	56	0
Test 5	4	118	187	56	0
Test 6	4	54	207	51	1
Test 7	4	56	185	55	0
Test 8	2	74	199	52	0
Test 9	3	55	191	57	1
Test 10	4	56	186	53	0
Average	3.3	66.1	191.9	55.7	0.4

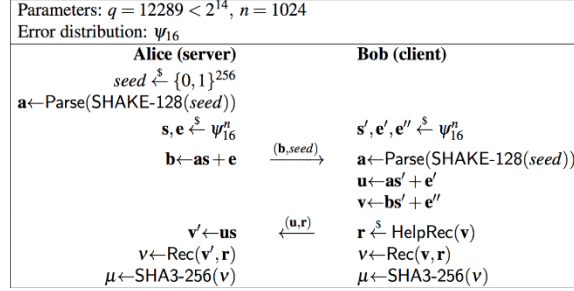
16384-bit key	DNS Lookup	TCP Connection	TLS Handshake	Server Processing	Content Transfer
Test 1	4	53	564	50	0
Test 2	4	60	621	58	0
Test 3	4	62	605	51	1
Test 4	2	52	502	54	0
Test 5	4	58	586	53	1
Test 6	4	53	551	54	1
Test 7	3	90	700	54	0
Test 8	4	68	529	51	1
Test 9	2	55	629	52	0
Test 10	2	53	603	53	1
Average	3.3	60.4	589	53	0.5

All times on these data tables are in milliseconds.

Some might perceive the increases in TLS transmission times shown above to be negligible. Between the 4096-bit key and the 8192-bit key, the increases in transmission times range from just 10-70 milliseconds when compared to the 2048-bit key standard. A seemingly fair trade when it comes to potentially discouraging quantum attacks. One issue that arises is that it is unknown to what extent the effectiveness of quantum systems in the near future, and RSA key exchanges aren't inherently resistant towards quantum calculations, only conventional ones.

The NewHope algorithm was introduced during the NIST post-quantum candidacy program. NewHope was built specifically to be quantum resistant by the use of the Ring Learning with Errors (RLWE). RLWE involves the transmission of the products between some randomly generated matrices with some secret matrix with a random noise distribution. After both endpoints transmit their products with their

respective secret matrix, both endpoints can generate a matching pair of symmetric keys to use for encrypting traffic between them. The simplistic representation of this process is shown below.



In the debut paper for NewHope, researchers justified this approach by stating that the Learning with Errors problem was proven to be as hard to solve under quantum reduction as the worst case for certain lattice problems.

Real implementations of NewHope are in their early stages, but we can see they contain promising results. Google recently added the NewHope key exchange method to some development branches of their web browser, Google Chrome. While not purely NewHope, Google implemented a hybrid protocol that combines tradition elliptic curve key exchange with NewHope, called CECPQ1, and even more recently an updated variant, CECPQ2. We made several calls to Google domains using both traditional protocols and CECPQ2 and recorded their results.

X25519	TLS Time(ms)	CECPQ2	TLS Time(ms)
Run 1	59	Run 1	72
Run 2	55	Run 2	71
Run 3	54	Run 3	68
Run 4	55	Run 4	74
Run 5	54	Run 5	68
Average	55.4	Average	70.6

X25519 is the traditional elliptic curve based key exchange method. As can be seen by the data, implementing NewHope on top of existing methods provides inherent quantum resistance at the cost of only about a 15 millisecond increase in key transmission times.

7 Benchmarking of Post Quantum Cryptography Algorithms

Our next part of the project focuses on benchmarking of various Quantum Cryptography algorithms. A great resource for this purpose is the Open Quantum Safe Project (OQS). OQS project aims to support the development and prototyping of quantum-resistant cryptography.

OQS consists of two main lines of work: liboqs, an open source C library for quantum-resistant cryptographic algorithms, and prototype integrations into protocols and applications, including the widely used OpenSSL library. Liboqs supports 7 PQ Key Encapsulation mechanisms (KEM) and 7 Signature

Schemes. It provided us with an opportunity to look closely at the performance of these algorithms.

Since each of these algorithms have 5-6 different versions, we focus our analyses on few chosen algorithms that cover all variants of PQCrypto and are hardware independent.

Thus, the algorithms we analyse are New Hope CCA (512,1024); NTRUEncrypt hps4096821; Kyber-1024; Frodo KEM AES (976,1344); SIKE (p751, p751 Compressed). We aim to study the trade-offs between key sizes and processing times.

All the tests were run on i7-7700K Kaby Lake processor Laptop, with Frequency of 4.20 GHz. All the algorithms were tested in their reference implementations i.e. C language.

Each algorithm's strength is determined by NIST Security Level. NIST specifies levels 1-5 of strength for Quantum Algorithms.

Level 1 is equivalent to or exceeds the brute force strength of AES 128 algorithm.

Each subsequent level is an increment of 32 bits, thus; Level 2: AES 160, Level 3: AES 192; Level 4: AES 224 and Level 5: AES 256.

Algorithm	Operation	Iterations	Mean Time (micro seconds)	Mean CPU Cycles	NIST Security Level	Secret Key (Bytes)	Public Key (Bytes)
New Hope 512 CCA	keygen	361,506	27.662	110,128	1	928	1888
	encaps	237,091	42.178	167,348			
	decaps	201,556	49.614	198,522			
New Hope 1024 CCA	keygen	168,339	59.404	238,126	5	3680	1824
	encaps	109,175	91.596	368,592			
	decaps	92,796	107.763	429,890			
FrodoKEM 976	keygen	1,485	673.484	2,580,364	3	31296	15332
	encaps	1,218	821.294	3,286,240			
	decaps	1,318	759.002	3,089,774			
FrodoKEM 1344	keygen	859	1163.702	4,556,722	5	43088	21520
	encaps	680	1471.396	5,873,148			
	decaps	696	1436.108	5,602,548			
NTRUEncrypt hps 4096821	keygen	577	1732.684	6,951,008	5	1592	1230
	encaps	9,752	102.548	410,196			
	decaps	3,870	258.396	1,049,032			
SIKE p751	keygen	8,939	1118.702	4,463,168	5*	644	564
	encaps	6,322	1581.568	6,291,482			
	decaps	5,120	1,953.07	7,795,180			
SIKE p751 Compressed	keygen	3,567	2803.751	11,203,758	5*	413	331
	encaps	2,899	3449.658	13,769,640			
	decaps	3,163	3162.254	12,627,716			
Kyber 1024	keygen	11,728	85.264	332,996	5	3168	1568
	encaps	10,075	99.256	397,004			
	decaps	5,888	169.838	673,442			

Following are the key observations:

- New Hope was implemented by Google in Chrome browsers, it is well supported by its reasonable key size and Processing times (~100 micro s).

- NTRU and Kyber although have key sizes in range of 1000s of bytes, provide really good times. NTRU has larger keygen time but it's encapsulation and decapsulation times are in the 100s of microseconds.
- Kyber-1024 posts better mean times in all three operations but it's secret key size is almost double that of NTRU and thrice that of New Hope.
- Frodo KEM is an Unstructured Lattice algorithm and has absurdly large key sizes, 15332 Bytes in Frodo 976 and 21520 Bytes in Frodo 1344.
- Their processing times for all three operations although much smaller than that of SIKE, do not compete with Structured Lattice Algorithms.
- The creators of Frodo KEM claim however that these times can be reduced by 40-60 times if implemented with AES accelerated hardware i.e., with AES NI Instructions enabled.
- SIKE is the solo entry in the Supersingular Isogeny (SI) group of algorithms. SIKE p751 has key sizes in the same range as RSA. (300-600 Bytes).
- The mean processing times for all three operations are significantly large. (1100-3500 micro s).
- The strength of SIKE p751 as mentioned by the authors is level 3, however recent researches have suggested that it may actually be higher upto Level 5.
- SI algorithms are based on Elliptical Curve Cryptography (ECC), which has troubled cryptographers with its own fair share of problems in the years gone by.
- Thus, SL algorithms seem to be the best option for implementation in TLS 1.3 Protocol suite.

Literature Review

1 Introduction

Serge Fehr introduces the concept of cryptography in his paper and builds into the introduction of quantum cryptography. In the quickest sense, classical cryptography is fundamentally flawed as the base that it stands on is not provably secure mathematics [1]. Current computing methods mean that cryptographic algorithms with high enough computational intensity for the quickest solving algorithms are deemed impenetrable. However, with the introduction of quantum computing, which allows for varied bit representation and therefore for a substantially greater number of simultaneous processes to coexist on a single machine. This nullifies the argument for protection by time as these processes can be completed in much more manageable time despite algorithmic and mathematical complexity. Therefore, algorithms that have been the standard for security must now be reevaluated and replaced to protect information despite the impending introduction of actual quantum computing into the world. Below are detailed a few methodologies and theories for re-securing information pending this implementation.

2 Quantum Computing

Problems that rely on computational intensity will eventually become obsolete when computing power eventually surpasses them. This is why quantum computing poses such a threat to cryptography. However, quantum mechanics comes with the boon of being reflected by nature. We can prove that a quantum cryptographic scheme can be secured via the law of nature. Examples of quantum cryptographic schemes are single polarized particles, and spin- $\frac{1}{2}$ particles [1]. Even if an attacker has complete access to the information, that attacker is still restricted to the information via the uncertainty principle [1]. The journal introduces ways to take on classic cryptographic tasks through the quantum and introduces quantum-key distribution (QKD) and two-party cooperation (2PC). QKD focuses on establishing secret keys in a public communication similar to Diffie-Hellman and 2PC focuses on security against inside attackers. The article also goes into the history of quantum computing from its roots to the attack on prime factorization which requires a quantum computer to defeat cryptographic schemes like RSA. Overall, the most important take from this paper is the ability to prove cryptographic schemes as secure instead of computationally intensive.

3 Public Key Distribution and Coin Tossing [2]

This paper introduces the application of key distribution with the addition of a quantum transmissions layered on top of conventional communication channels. One-time pad is an existing encryption method that depends on the secret transmission of a single-use, private key between two endpoints to achieve. In a

conventional communication channel vulnerable to eavesdropping, the secrecy of this key transmission is not guaranteed. In order to realize this private key transmission, the quantum system becomes useful. One endpoint will begin by generating a random string of bits and a series of random bases, where each bit and each base in their respective position will form a “pair”. This same endpoint will then broadcast photons on the quantum channel, where each photon is specifically oriented in relation to the pair value that was randomly generated. The other endpoint will then receive these photons and will randomly generate a subset of the photons, measuring their polarization and determining whether their values represent a logical 1 or 0. Back on the conventional channel, these two endpoints will determine publicly which photons were received, and which of those received had their value determined correctly. By the nature of the quantum transmission, disturbing these photons during delivery to the endpoint would result in the alteration of the data they represent. This results in the two endpoints not being able to determine which of the data represented by polarized photons were processed correctly, and the transmission would have to be attempted again. This theoretically results in a private transmission of a key across a “public” channel.

4 Post-Quantum Key Exchange [3]

This paper defines New Hope, which is a key-exchange protocol based on the Ring-Learning-with-Errors (Ring-LWE) problem. It selected by Google for an experiment to reduce the impact of quantum computers. The method is named “Ring Learning With Errors” (Ring-LWE) and is a key exchange method which would be robust in the age of quantum computing. Ring-LWE currently has no known weaknesses within a quantum computing era. Their first agenda is to specify the vulnerabilities in established public key encryption methods. Section 2.3 introduces the New Hope proposal. New Hope offers a 256-bit security level, has performance advantages over previous work due to the use of a better suited error distribution, a new reconciliation mechanism, efficient defense against backdoors and so-called “all-for-the-price-of-one” attacks. Complete description and C,AW2 implementations of the algorithm are detailed. They conclude that small chip size and limited storage space for storing and executing such a complex algorithm, as well as the transmission speed were challenging, but puts them in a leading position in this field of encryption that withstands quantum computing power. Their results show that their proposed parameters offer at least (and quite likely with a large margin) a post-quantum security of 128 bits. This superseded the BCNS and RSA/ECDH systems. As the prevalent and most advanced PQC algorithm, New Hope marks the genesis of Quantum Safe Cryptography.

5 Crypto-agility Strategies [4]

The authors of this publication have given an in-depth review of strategies that must be employed to ward off possible attacks on established mechanisms using quantum computers and algorithms. In section 2 of the paper, they provide a short description

of notable quantum algorithms, Shor's algorithm and Grover's algorithm and provide an explanation of their increased efficiency. The next section deals with the concept of crypto-agility, i.e. the ability of established cryptography systems to adapt to PQC primitives. Popular algorithms namely, AES, RSA, ECC, SIDH and Hash-based Cryptography (SHA-2/-3) have been studied and their security against and resilience to quantum computing has been determined. They present candidates for quantum-safe primitives such as Multivariate Cryptography, Grid-based Cryptography, Coding-based Cryptography and Isogeny-based Cryptography with detailed algorithms. Examples of quantum-based implementation of security protocols such as New Hope by Google have been introduced. In their results they classify symmetric encryption as quantum resistant, whereas asymmetric encryption schemes are found to be prone to quantum attacks. They state that public key methods must be adapted to PQC approved algorithms to ensure quantum security. This paper is an essential resource to us as the basis of Post Quantum Cryptography as it provides useful information regarding quantum resilience of popular algorithms and prominent PQC implementation techniques.

6 Cryptography: Union of Cryptography and Steganography

This paper defined cryptography as a combinatory discipline between cryptography and steganography [5]. The paper encourages the use of both Multivariate Public Key Cryptography and Steganography in order to add two methods of slowing access to encrypted data. The primary revolutionary argument here is the use of Steganography to disguise what information is encrypted and worth decrypting from those attempting to find and decipher important or potentially damaging information. Steganography uses unremarkable cover media to disguise the presence of desirable data within the transmitted data [5]. The theory behind this is to substantially diminish the probability of an attempt to decipher the coveted data, as opposed to the standard strategy of assuming an attack will be attempted. In this way, increasingly effective steganography techniques are impervious to the advancements of quantum computing, as they are broken more by curiosity than by increased speed of computation. The addition of MPKC to computationally strengthen the already hidden data provides another, though not impervious, barrier to deciphering.

References

- [1] Fehr, S. (2010). Quantum Cryptography. *Foundations of Physics*, 40(5), pp.494-531.
- [2] Bennett H. Charles , Brassard Gilles .”Quantum cryptography: Public key distribution and coin tossing”. 4 December 2014, BM Research, Yorktown Heights NY 10598, USA.
- [3] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, Peter Schwabe “Post-Quantum Key Exchange—A New Hope.” *25th USENIX Security Symposium USENIX Security 16*, August 2016, 327–43.
- [4] Grote, Olaf, Andreas Ahrens, and Cesar Benavente-Peces. “A Review of Post-Quantum Cryptography and Crypto-Agility Strategies.” *2019 International Interdisciplinary PhD Workshop (IIPhDW)*, 2019. <https://doi.org/10.1109/iiphdw.2019.8755433>
- [5] A. J. Gabriel, B. K. Alese, A. O. Adetunmbi and O. S. Adewale, "Post-Quantum Cryptography: A combination of Post-Quantum Cryptography and Steganography," *8th International Conference for Internet Technology and Secured Transactions (ICITST-2013)*, London, 2013, pp. 449-452.

Investigator Biographies

Ali Sbeiti

Sbeiti is currently enrolled in The University of Texas at Dallas fast track program, pursuing a Master's degree in Computer Science under the Information Assurance track. His interests include: participating in ethical hacking challenges, exploring issues in network security, and discovering new solutions and technologies. He is proficient in C, C++, Java, Python, Javascript, and PHP. Relevant past courses include: Computer Networks, Network Security, and Data and application Security. His goals are to discover the limitations of modern security solutions being implemented today and be a part of newer solutions.

Troy Frazier

Frazier is a fast track Computer Science student pursuing a Master's in Information Assurance. He has professional experience with forensic analysis, cyber security, and augmented reality through various internships. He loves cryptography and network security. He has coursework knowledge of various security concepts such as networking, malware, botnets, cryptography, buffer overflow, and much more. His main programming language is the C family (C, C++, C#), but can easily do any imperative programming language and has stable foundations within functional/logical languages. His goal is to advance cryptography in preparation of the next age and he desires to work for the government.

Samantha Gant

Gant is a fast track Computer Science student pursuing a Master's degree in Information Assurance. She has professional experience with messaging, remote data access, and some penetration testing as a result of internships. She is passionate about cryptography and information security. Her main programming languages are C/C++ and Java, but she has experience with logical and functional languages as well. Her goal is to take a deeper dive into real world cryptography and security issues to improve present and future security and grow a better understanding of the problems that do and will exist as present technologies come into greater use.

Aviral Verma

Verma holds a Bachelor's degree in Computer Science and is currently pursuing a Master's in Information Assurance. He is passionate about cryptography and has undergone certified Ethical Hacking Training. His undergraduate coursework has helped him learn the fundamentals of computer networks, cryptography and cloud security. He is proficient in Python, C, C++ and Java programming languages. He

has interned at IBM India and Krgyion Tech participating in cyber security-based projects, primarily focused on cryptography and network security. He has experience in Android App Development and is skillful in Algorithms Design. He hopes to expand his knowledge and sharpen his skills to help the advancement of Cyber Security.